
Private Token Sale Smart Contract: Function Overview and Execution Guide

1. Contract Functions

Constructor

Purpose:

Initializes the contract by setting the deployer as the owner. This ensures that only the deployer has privileged control over specific functions.

Function Signature:

```
constructor() public
```

- **Key Point:** Only the deployer will have ownership until transferred.
-

2. startSale()

- **Purpose:**
Starts the private token sale and can only be called by the contract owner.

Event Emitted: `SaleStatusChanged(true)`

- **Key Point:** Sale can only be initiated by the current owner.
-

3. endSale()

- **Purpose:**
Ends the private token sale and restricts access to the contract owner.
 - **Event Emitted:** `SaleStatusChanged(false)`
 - **Key Point:** Prevents further purchases after the sale is closed.
-

4. `getTokenDecimals(address token)`

- **Purpose:**
Fetches the number of decimals for a specific ERC20 token by calling the token's `decimals()` function.

Functionality: Helps normalize token values when interacting with different tokens that use different decimal places.

- **Key Point:** Ensures dynamic handling of token precision.
-

5. `BuyWithUSDC(uint256 amount)`

- **Purpose:**
Allows a user to participate in the token sale by purchasing with USDC. The function transfers the specified amount from the user's wallet to the contract and updates the user's USDC deposit.

Event Emitted: `Deposit`

- **Key Point:** Users must approve the contract to handle their USDC before purchasing.
-

6. `BuyWithDAI(uint256 amount)`

- **Purpose:**
Similar to the USDC function, this allows users to purchase tokens using DAI. The amount is transferred from the user to the contract, updating their DAI deposit.

Event Emitted: `Deposit`

- **Key Point:** Users must first approve the contract to handle their DAI.
-

7. BuyWithUSDP(uint256 amount)

- **Purpose:**
Enables token purchases with USDP. The function transfers the amount of USDP to the contract, updating the user's USDP deposit.

Event Emitted: `Deposit`

- **Key Point:** Users must first approve USDP transfers.
-

8. getAllocatedTokens(address user)

- **Purpose:**
Returns the total stablecoin deposits (USDC, USDP, DAI) for a given user.

- **Key Point:** Useful for users to track their investments in the private sale.
-

9. withdrawTokens()

- **Purpose:**
Allows the contract owner to withdraw all the collected stablecoins (USDC, USDP, DAI) to their wallet.

- **Key Point:** Only the owner has access to these funds after the sale.
-

10. transferOwnership(address _newOwner)

- **Purpose:**
Transfers ownership of the contract to a new owner.

Restrictions:

- Can only be called by the current owner.
 - The new owner cannot be the zero address (0x0).
 - **Key Point:** This function ensures secure delegation of ownership.
-

11. withdrawERC20(address stuckTokenAddress)

- **Purpose:**
Enables the owner to withdraw any ERC20 tokens (other than USDC, USDP, or DAI) that may be stuck in the contract.
 - **Key Point:** Helps recover accidentally sent tokens.
-

12. withdrawStuckETH()

- **Purpose:**
Allows the owner to withdraw any Ether (ETH) stuck in the contract.
- **Key Point:** Prevents Ether from being permanently locked in the contract.

Instructions & Steps to Execute

1. Deploy the Contract

- **Step:**
Deploy the **PrivateSale** contract on the Ethereum network.
 - **Result:** The deployer becomes the owner of the contract.
-

2. Start the Sale

- **Step:**

The owner calls the `startSale()` function.

- **Result:** The private sale is activated, and users can begin purchasing tokens.
-

3. Approve Token Transfer

- **Step:**

Users must approve the contract to transfer their stablecoins (USDC, USDP, or DAI) on their behalf.

Example:

```
ERC20(USDC).approve(privateSaleAddress, amount);
```

-

- **Result:** The contract can transfer tokens from the user's wallet.
-

4. Participate in the Sale

- **Step:**

Users can purchase tokens by calling one of the following functions, depending on their stablecoin:

- **BuyWithUSDC(uint256 amount)** – Purchase with USDC.
 - **BuyWithDAI(uint256 amount)** – Purchase with DAI.
 - **BuyWithUSDP(uint256 amount)** – Purchase with USDP.
 - **Result:** The user's deposit is recorded, and the stablecoins are transferred to the contract.
-

5. Check User Deposits

- **Step:**

Users can call the `getAllocatedTokens(userAddress)` function to check their total deposits.

- **Result:** The function returns the user's total stablecoin deposits in the contract.
-

6. End the Sale

- **Step:**
The owner calls the `endSale()` function to deactivate the sale.
 - **Result:** The sale ends, and no further purchases can be made.
-

7. Withdraw Stablecoins

- **Step:**
Once the sale ends, the owner can call `withdrawTokens()` to withdraw all collected stablecoins.
 - **Result:** Stablecoins (USDC, USDP, DAI) are transferred to the owner's wallet.
-

8. Transfer Ownership

- **Step:**
If the owner wishes to transfer ownership, they can call the `transferOwnership(newOwnerAddress)` function.
 - **Result:** Ownership is transferred to the new address.
-

9. Handle Stuck Tokens and ETH

- **Step:**
The owner can:
 - Retrieve any ERC20 tokens stuck in the contract by calling `withdrawERC20(stuckTokenAddress)`.
 - Withdraw any Ether (ETH) stuck in the contract by calling `withdrawStuckETH()`.
 - **Result:** The owner ensures no tokens or ETH are permanently stuck in the contract.
-

Link to code -

<https://github.com/PandemicCoinPDC/PDC-smart-contracts/blob/main/Tokensale.sol>